Tietokantojen perusteet, syksy 2026, projektitehtävät

Opiskelijan nimi: Tomi K

Opiskelijanumero:

Projektitehtävien yhteispistemäärä: /100 pistettä

Tehtävä 1

Tehtävän pistemäärä: /30 pistettä

Osatehtävä 1

Toteutettu koodi:

```
import sqlite3
import sys

db = sqlite3.connect("bikes_2024.db")

def trips_from_station(station_name):
    sql = """SELECT COUNT(*)
             FROM Trips AS T
             JOIN Stations AS S ON T.start_station_id = S.id
             WHERE S.name = ?"""
    result = db.execute(sql, [station_name])
    return result.fetchone()[0]

def trips_to_station(station_name):
    sql = """SELECT COUNT(*)
             FROM Trips AS T
             JOIN Stations AS S ON T.end_station_id = S.id
             WHERE S.name = ?"""
    result = db.execute(sql, [station_name])
    return result.fetchone()[0]

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python main.py <station_name>")
        sys.exit(1)

    station_name = sys.argv[1]

    print(f"Asemalta {station_name} alkaneiden matkojen määrä on: "
          f"{trips_from_station(station_name)}")
    print(f"Asemalle {station_name} päättyneiden matkojen määrä on: "
          f"{trips_to_station(station_name)}")

    db.close()
```

Osatehtävä 2

Toteutettu koodi:

```
import sqlite3
import sys

db = sqlite3.connect("bikes_2024.db")

def longest_distance_trips(limit):
    sql = """SELECT S1.name, S2.name, T.distance
             FROM Trips AS T
             JOIN Stations AS S1 ON T.start_station_id = S1.id
             JOIN Stations AS S2 ON T.end_station_id = S2.id
             ORDER BY T.distance DESC
             LIMIT ?"""
    result = db.execute(sql, [limit])
    return result.fetchall()

def longest_duration_trips(limit):
    sql = """SELECT S1.name, S2.name, T.duration
             FROM Trips AS T
             JOIN Stations AS S1 ON T.start_station_id = S1.id
             JOIN Stations AS S2 ON T.end_station_id = S2.id
             ORDER BY T.duration DESC
             LIMIT ?"""
    result = db.execute(sql, [limit])
    return result.fetchall()

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python ex1_2.py <limit>")
        sys.exit(1)

    limit = int(sys.argv[1])

    print("Pisin matkaetäisyys on seuraavilla matkoilla:")
    for start_station, end_station, distance in longest_distance_trips(limit):
        print(f"{start_station:<25} => {end_station:<25} = {distance} m")

    print("\nPisin kestoiset matkat ovat:")
    for start_station, end_station, duration in longest_duration_trips(limit):
        print(f"{start_station:<25} => {end_station:<25} = {duration} s")

    db.close()
```

Osatehtävä 3

Toteutettu koodi:

```
import sqlite3
import sys

db = sqlite3.connect("bikes_2024.db")

months = [
    "Tammikuu", "Helmikuu", "Maaliskuu", "Huhtikuu",
    "Toukokuu", "Kesäkuu", "Heinäkuu", "Elokuu",
    "Syyskuu", "Lokakuu", "Marraskuu", "Joulukuu"
]

weekdays = [
    "Maanantai", "Tiistai", "Keskiviikko", "Torstai",
    "Perjantai", "Lauantai", "Sunnuntai"
]

def trips_during_month(month):
    sql = """SELECT COUNT(*)
             FROM Trips
             WHERE CAST(strftime('%m', start_time) AS INTEGER) = ?"""
    result = db.execute(sql, [month])
    return result.fetchone()[0]

def trips_during_weekday(day):
    sql = """SELECT COUNT(*)
             FROM Trips
             WHERE (CAST(strftime('%w', start_time) AS INTEGER) + 6)
                    % 7 + 1 = ?"""
    result = db.execute(sql, [day])
    return result.fetchone()[0]

if __name__ == "__main__":
    if len(sys.argv) < 3:
        print("Usage: python ex1_3.py <month> <weekday>")
        sys.exit(1)

    month = (int(sys.argv[1]) - 1) % 12 + 1
    day = (int(sys.argv[2]) - 1) % 7 + 1

    print(f"{months[month - 1]}n matkojen määrä on: "
          f"{trips_during_month(month)}")

    print(f"{weekdays[day - 1]}n matkojen määrä on: "
          f"{trips_during_weekday(day)}")

    db.close()
```

Osatehtävä 4

Toteutettu koodi:

```
import sqlite3
import sys

db = sqlite3.connect("bikes_2024.db")

def most_popular_start_stations(limit):
    sql = """SELECT S.name, COUNT(*) AS departures
             FROM Trips AS T
             JOIN Stations AS S ON T.start_station_id = S.id
             GROUP BY S.id
             ORDER BY departures DESC
             LIMIT ?"""
    result = db.execute(sql, [limit])
    return result.fetchall()

def least_popular_start_stations(limit):
    sql = """SELECT S.name, COUNT(*) AS departures
             FROM Trips AS T
             JOIN Stations AS S ON T.start_station_id = S.id
             GROUP BY S.id
             ORDER BY departures ASC
             LIMIT ?"""
    result = db.execute(sql, [limit])
    return result.fetchall()

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python ex1_4.py <limit>")
        sys.exit(1)

    limit = int(sys.argv[1])

    print("Suosituimmat lähtöasemat ovat:")
    for value in most_popular_start_stations(limit):
        print(f"{value[0]}: {value[1]}")

    print("\nVähiten suositut lähtöasemat ovat:")
    for value in least_popular_start_stations(limit):
        print(f"{value[0]}: {value[1]}")

    db.close()
```

Osatehtävä 5

Toteutettu koodi:

```
import sqlite3
import sys

db = sqlite3.connect("bikes_2024.db")

def largest_differences(limit):
    sql = """WITH departures AS (
                SELECT start_station_id, COUNT(*) AS starting
                FROM Trips
                GROUP BY start_station_id
            ),
            arrivals AS (
                SELECT end_station_id, COUNT(*) AS ending
                FROM Trips
                GROUP BY end_station_id
            )
            SELECT S.name,
                   IFNULL(D.starting, 0),
                   IFNULL(A.ending, 0),
                   ABS(IFNULL(D.starting, 0) -
                       IFNULL(A.ending, 0)) AS difference
            FROM Stations AS S
            LEFT JOIN departures AS D ON S.id = D.start_station_id
            LEFT JOIN arrivals AS A ON S.id = A.end_station_id
            ORDER BY difference DESC
            LIMIT ?"""
    result = db.execute(sql, [limit])
    return result.fetchall()

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python ex1_5.py <limit>")
        sys.exit(1)

    limit = int(sys.argv[1])

    print("Suurimmat erot alkaneiden ja päättyneiden matkojen määrässä:")
    print(
        f"{'Asema':<30} {'Alkaneet matkat':>15} "
        f"{'Päättyneet matkat':>18} {'Määrien ero':>12}"
    )

    for value in largest_differences(limit):
        print(f"{value[0]:<30} {value[1]:>15} {value[2]:>18} {value[3]:>12}")

    db.close()
```

Tehtävä 2

Tehtävän pistemäärä: /10 pistettä

SQL-skeema:

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS Users (
    id INTEGER PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS Listings (
    id INTEGER PRIMARY KEY,
    seller_id INTEGER NOT NULL REFERENCES Users(id),
    title TEXT NOT NULL,
    description TEXT NOT NULL,
    price REAL NOT NULL CHECK (price >= 0),
    status TEXT NOT NULL DEFAULT 'available'
        CHECK (status IN ('available', 'reserved', 'sold')),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS Images (
    id INTEGER PRIMARY KEY,
    listing_id INTEGER NOT NULL UNIQUE REFERENCES Listings(id)
        ON DELETE CASCADE,
    image BLOB NOT NULL
);

CREATE TABLE IF NOT EXISTS Categories (
    id INTEGER PRIMARY KEY,
    type TEXT NOT NULL,
    value TEXT NOT NULL,
    UNIQUE (type, value)
);

CREATE TABLE IF NOT EXISTS ListingCategories (
    listing_id INTEGER NOT NULL REFERENCES Listings(id)
        ON DELETE CASCADE,
    category_id INTEGER NOT NULL REFERENCES Categories(id),
    PRIMARY KEY (listing_id, category_id)
);

CREATE TABLE IF NOT EXISTS Favorites (
    user_id INTEGER NOT NULL REFERENCES Users(id)
        ON DELETE CASCADE,
    listing_id INTEGER NOT NULL REFERENCES Listings(id)
        ON DELETE CASCADE,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, listing_id)
```

```
    UNIQUE(user_id, listing_id)
);

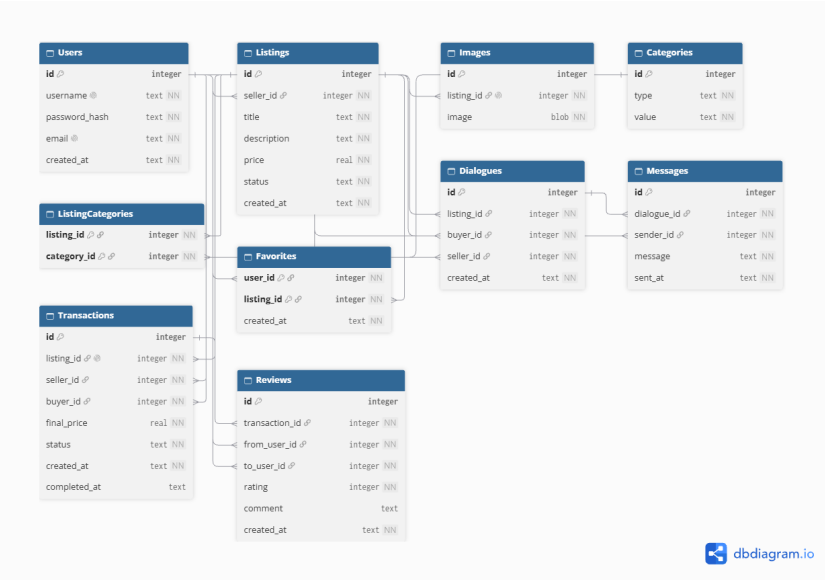
CREATE TABLE IF NOT EXISTS Dialogues (
    id INTEGER PRIMARY KEY,
    listing_id INTEGER NOT NULL REFERENCES Listings(id),
    buyer_id INTEGER NOT NULL REFERENCES Users(id),
    seller_id INTEGER NOT NULL REFERENCES Users(id),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (listing_id, buyer_id)
);

CREATE TABLE IF NOT EXISTS Messages (
    id INTEGER PRIMARY KEY,
    dialogue_id INTEGER NOT NULL REFERENCES Dialogues(id)
        ON DELETE CASCADE,
    sender_id INTEGER NOT NULL REFERENCES Users(id),
    message TEXT NOT NULL,
    sent_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS Transactions (
    id INTEGER PRIMARY KEY,
    listing_id INTEGER NOT NULL UNIQUE
        REFERENCES Listings(id),
    seller_id INTEGER NOT NULL REFERENCES Users(id),
    buyer_id INTEGER NOT NULL REFERENCES Users(id),
    final_price REAL NOT NULL CHECK (final_price >= 0),
    status TEXT NOT NULL DEFAULT 'reserved'
        CHECK (status IN ('reserved', 'completed', 'cancelled')),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    completed_at TEXT
);

CREATE TABLE IF NOT EXISTS Reviews (
    id INTEGER PRIMARY KEY,
    transaction_id INTEGER NOT NULL REFERENCES Transactions(id),
    from_user_id INTEGER NOT NULL REFERENCES Users(id),
    to_user_id INTEGER NOT NULL REFERENCES Users(id),
    rating INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment TEXT,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (transaction_id, from_user_id)
);
```

Tietokantakaavio:



Toimintojen selostukset:

1. Käyttäjä voi kirjautua järjestelmään antamalla tunnuksen ja salasanan. Tauluun Users tallentuvat käyttäjän tunnus ja salasanan hash (username, password_hash).
2. Käyttäjä pystyy katsomaan muiden käyttäjien lähettämiä ilmoituksia sekä lähettämään omia ilmoituksia. Tauluun Listings tallentuvat kaikki ilmoitukset, jotka ovat näkyvissä kaikille. Kun käyttäjä itse luo ilmoituksen, niin tauluun tallentuu hänen User ID.
3. Ilmoituksessa näkyy myytävän tavaran tiedot (otsikko, kuvaus, hinta) sekä yksi tai useampia kuvia. Listings taulussa on nämä tieto kentät (title, description, price) ja kuvat löytyvät Images taulusta joka liittyy Listings taulun ID:hen.
4. Ilmoituksessa voi olla luokittelu myytävälle tavaralle (esimerkiksi osasto kirjat, sijainti Helsinki). On olemassa Categories taulu jossa on ennaltamääritettyjä luokkia. ListingCategories taulu liittää nämä Listings tauluun.
5. Käyttäjä voi merkitä ilmoituksen suosikikseen. Tämä onnistuu ja tallentuu Favorites tauluun, joka liittyy yksittäiseen käyttäjään ja listaukseen. Taulujen (Users ja Listings) ID:hin ja on uniikki.
6. Kaksi käyttäjää pystyy viestittelemään keskenään ilmoitukseen liittyen. Tämä onnistuu Dialogues ja Messages taulujen välityksellä. Dialogues on keskusteluketju, ja Messages sisältää keskustelun yksittäiset viestit. Dialogues liittyy ilmoitukseen ja kahteen käyttäjään, kun taas Messages liittyy Dialogues tauluun (keskusteluketju) ja yk-

sittäiseen käyttäjään.
7. Käyttäjä pystyy merkitsemään tavaran varatuksi ja myydyksi. Myyjä voi muuttaa tavaran tilaa Listings-taulun status-sarakkeen avulla. Tavara voidaan myös merkitä automaattisesti myydyksi, jos se on huutokaupassa ja saavutettu tarjous täyttää asetetun minimihinnan. Jos tavaraa ei saada myytyä tietyn ajan kuluessa, sen tila voidaan mahdollisesti muuttaa automaattisesti varatuksi tai myyjä voi itse merkitä tavaran varatuksi.
8. Tavaran myynnin jälkeen myyjä ja ostaja pystyvät antamaan toisilleen palautteen. Palautteen antaminen on mahdollista, kun huutokauppajärjestelmä on tallentanut myyntitapahtuman Transactions tauluun ja merkinnyt sen suoritetuksi. Tämän jälkeen sekä ostaja että myyjä voivat antaa toisilleen palautetta, joka tallennetaan Reviews-tauluun. Transaktio liittyy kahteen käyttäjään ja listaukseen ja palaute liittyy Transaction tauluun ja kumpaankin käyttäjään. Käyttäjät voivat antaa toisilleen luokituksen ja jättää kommentteja.

Tehtävä 3

Tehtävän pistemäärä: /10 pistettä

Test 1

- Minkä tuloksen 1. transaktion SELECT-kysely antaa? 1
- Minkä tuloksen 2. transaktion SELECT-kysely antaa? 2
- Onnistuuko 1. transaktio? Kyllä.
- Onnistuuko 2. transaktio? Ei.
- Miten perustelet nämä tulokset? K2:n päivitys onnistuu, koska K1:llä ei ole kirjoituslukkoa. K2:n COMMIT kuitenkin epäonnistuu, koska K1:n asettama lukulukko estää transaktion vahvistamisen. K1:n COMMIT onnistuu, koska se ei ole tehnyt muutoksia. Tämän jälkeen K2 voisi suorittaa COMMIT-komennon onnistuneesti

Test 2

- Minkä tuloksen 1. transaktion SELECT-kysely antaa? 1
- Minkä tuloksen 2. transaktion SELECT-kysely antaa? 1
- Onnistuuko 1. transaktio? Kyllä.

- Onnistuuko 2. transaktio? Kyllä.
- Miten perustelet nämä tulokset? Tässä testissä tapahtumien järjestys on suotuisampi kuin edellisessä testissä. K1 ei ole vielä vahvistanut muutostaan COMMIT-komennolla, joten K2 ei näe sitä. K2:n COMMIT onnistuu, koska se ei ole tehnyt muutoksia tietokantaan. Tämän jälkeen K2:n asettama lukulukko vapautuu, minkä ansiosta myös K1:n COMMIT onnistuu.

Test 3

- Minkä tuloksen 1. transaktion SELECT-kysely antaa? 2
- Minkä tuloksen 2. transaktion SELECT-kysely antaa? 1
- Onnistuuko 1. transaktio? Kyllä.
- Onnistuuko 2. transaktio? Ei.
- Miten perustelet nämä tulokset? K1 päivittää arvon 2, mutta ei vielä kirjoita muutosta. K2 yrittää päivittää arvon 3, mutta epäonnistuu koska K1:llä on kirjoituslukko. K2:n SELECT siten palauttaa 1, koska K1:n muutos ei ole vielä tallentunut. K2:n COMMIT onnistuu, koska aiempi päivitys epäonnistui eikä ole tallennettavia muutoksia. K1:n SELECT palauttaa 2, koska transaktio näkee omat muutoksensa. K1:n COMMIT onnistuu, koska K2 vapautti lukulukon. Jos K2 ei olisi ajanut COMMIT komentoa, niin ei K1 pystyisi tallentamaan muutoksiaan.

Test 4

- Minkä tuloksen 1. transaktion SELECT-kysely antaa? 2
- Minkä tuloksen 2. transaktion SELECT-kysely antaa? 1
- Onnistuuko 1. transaktio? Kyllä.
- Onnistuuko 2. transaktio? Ei.
- Miten perustelet nämä tulokset? Tilanne on täysin sama kuin edellisessä testissä. Ainoa ero on, että K2 yrittää päivittää arvon arvoon 2 arvon 3 sijaan. Edellä esitetyt kommentit testissä 3 pätevät myös tähän tapaukseen.

Test 5

- Minkä tuloksen 1. transaktion SELECT-kysely antaa? 1
- Minkä tuloksen 2. transaktion SELECT-kysely antaa? 1
- Onnistuuko 1. transaktio? Ei.
- Onnistuuko 2. transaktio? Ei.
- Miten perustelet nämä tulokset? K1 ja K2 kumpikin lukevat arvon 1. K2 päivittää arvon 2. K1:n lukulukko estää K2:sen muutosten vahvistamisen ja COMMIT epäonnistuu. K1 yrittää päivittää arvon 2, mutta K2:n kirjoituslukko estää sen ja UPDATE epäonnistuu. K1:n COMMIT onnistuu, koska aiempi päivitys epäonnistui eikä K1:llä ole tallennettavia muutoksia.

Tehtävä 4

Tehtävän pistemäärä: /10 pistettä

Toteutettu koodi:

```
import sqlite3
import random
import string
import time
import os

def create_movies(db):
    db.execute("DROP TABLE IF EXISTS Movies")
    db.execute("""CREATE TABLE Movies (
                      id INTEGER PRIMARY KEY,
                      name TEXT,
                      release_year INTEGER)""")

def insert_movies(db):
    movies = []

    for _ in range(1000000):
        title = "".join(random.choices(string.ascii_letters, k=8))
        year = random.randint(1900, 2000)
        movies.append((title, year))

    start = time.perf_counter()

    db.execute("BEGIN")
    db.executemany(
        "INSERT INTO Movies (name, release_year) VALUES (?, ?)", movies
    )
    db.execute("COMMIT")
```

```
    return time.perf_counter() - start

def query_movies(db):
    years = [random.randint(1900, 2000) for _ in range(1000)]

    start = time.perf_counter()

    for year in years:
        db.execute(
            "SELECT COUNT(*) FROM Movies WHERE release_year = ?", (year,)
        ).fetchone()

    return time.perf_counter() - start

def test_without_index():
    filename = "movies1.db"
    db = sqlite3.connect(filename)
    create_movies(db)
    insert_time = insert_movies(db)
    query_time = query_movies(db)
    db.close()
    size = os.path.getsize(filename)

    return insert_time, query_time, size

def test_with_index_before_insert():
    filename = "movies2.db"
    db = sqlite3.connect(filename)
    create_movies(db)
    db.execute(
        "CREATE INDEX idx_release_year ON Movies(release_year)"
    )
    insert_time = insert_movies(db)
    query_time = query_movies(db)
    db.close()
    size = os.path.getsize(filename)

    return insert_time, query_time, size

def test_with_index_before_queries():
    filename = "movies3.db"
    db = sqlite3.connect(filename)
    create_movies(db)
    insert_time = insert_movies(db)
    db.execute(
        "CREATE INDEX idx_release_year ON Movies(release_year)"
    )
    query_time = query_movies(db)
    db.close()
    size = os.path.getsize(filename)

    return insert_time, query_time, size

if __name__ == "__main__":
```

```
results = []

results.append(("Ei indeksiä", test_without_index()))
results.append(("Indeksi ennen lisäystä", test_with_index_before_insert()))
results.append(("Indeksi ennen kyselyitä", test_with_index_before_queries()))

for name, (insert_time, query_time, size) in results:
    print(f"{name}:")
    print(f"Rivien lisäys: {insert_time:.3f} s")
    print(f"Kyselyt:       {query_time:.3f} s")

for name, (insert_time, query_time, size) in results:
    print(f"{name}: {size / 1000000:.2f} MB")
```

- Testin 1 tulokset:
  - Rivien lisäämiseen kuluva aika (sekunteina) 1.092
  - Kyselyiden suoritukseen kuluva aika (sekunteina) 93.013
  - Tietokantatiedoston koko testin lopuksi (megatavuina) 20.12
- Testin 2 tulokset:
  - Rivien lisäämiseen kuluva aika (sekunteina) 2.411
  - Kyselyiden suoritukseen kuluva aika (sekunteina) 0.906
  - Tietokantatiedoston koko testin lopuksi (megatavuina) 32.47
- Testin 3 tulokset:
  - Rivien lisäämiseen kuluva aika (sekunteina) 1.234
  - Kyselyiden suoritukseen kuluva aika (sekunteina) 0.941
  - Tietokantatiedoston koko testin lopuksi (megatavuina) 31.15
- Miten selität testin tulokset? Indeksi nopeuttaa kyselyiden suorittamista noin 100-kertaisesti, koska sen avulla tietokanta voi löytää hakuehtoja vastaavat rivit ilman, että koko taulukkoa tarvitsee käydä läpi. Jos indeksi luodaan ennen rivien lisäämistä, lisäysoperaatio hidastuu, koska indeksiä on päivitettävä jokaisen uuden rivin yhteydessä. Indeksi kasvatti myös tietokantatiedoston kokoa noin 10 megatavulla. Vaikuttaisi että indeksistä on eniten hyötyä silloin, kun tietokannasta tehdään paljon hakuja. Sen hyöty voi jäädä vähäiseksi, jos kysely palauttaa suuren osan taulukon riveistä.

Tehtävä 5

Tehtävän pistemäärä: /10 pistettä

Toteutettu koodi:

```
class Relation:
    def __init__(self, attributes):
        self.attributes = tuple(attributes)
        self.tuples = set()

    def add_tuple(self, values):
        self.tuples.add(tuple(values))

    def __str__(self):
        return str(self.tuples)

def projection(relation, attributes):
    indexes = [relation.attributes.index(a) for a in attributes]

    result = Relation(attributes)
    result.tuples = {
        tuple(row[i] for i in indexes)
        for row in relation.tuples
    }

    return result

def restriction(relation, attribute, value):
    index = relation.attributes.index(attribute)

    result = Relation(relation.attributes)
    result.tuples = {
        row for row in relation.tuples
        if row[index] == value
    }

    return result

if __name__ == "__main__":
    products = Relation(("id", "name", "price"))

    products.add_tuple((1, "retiisi", 7))
    products.add_tuple((2, "porkkana", 5))
    products.add_tuple((3, "nauris", 4))
    products.add_tuple((4, "lanttu", 8))
    products.add_tuple((5, "selleri", 4))

    print(products)

    print(projection(products, ("name",)))
    print(projection(products, ("price",)))
    print(projection(products, ("name", "price")))

    print(restriction(products, "name", "porkkana"))
```

```
    print(restriction(products, "price", 4))

    print(projection(restriction(products, "price", 4), ("name",)))
```

Tehtävä 6

Tehtävän pistemäärä: /10 pistettä

Toteutettu koodi:

```
import pymongo

connection_string = "mongodb+srv://tikape:NAq8a4pNLWF8TMfd@cluster0.u4vehy9.mongodb.net/"
client = pymongo.MongoClient(connection_string)

database = client.get_database("tikape")
apartments = database["apartments"]

# 1. Monessako asunnossa postinumero on 00700?
count_postal_code = apartments.count_documents({
    "zip_code": "00700"
})
print("Asunnot postinumerolla 00700:", count_postal_code)

# 2. Monessako asunnossa rakennusvuosi on 2000-luvulla?
count_construction_year = apartments.count_documents({
    "construction_year": {"$gte": 2000, "$lte": 2099}
})
print("2000-luvulla rakennetut asunnot:", count_construction_year)

# 3. Monessako asunnossa pinta-ala on välillä 50{70 m²?
count_apartment_size = apartments.count_documents({
    "apartment_size": {"$gte": 50, "$lte": 70}
})
print("Pinta-alaltaan 50-70 m²:n asunnot:", count_apartment_size)

# 4. Moniko asunnoista on myyty ainakin kerran vuosina 2010{2012?
count_transactions = apartments.count_documents({
    "transactions.date": {"$gte": "2010-01-01", "$lt": "2013-01-01"}
})
print("Vuosina 2010-2012 myydyt asunnot:", count_transactions)

# 5. Mikä on kallein asunnon myyntihinta aineistossa?
result = apartments.aggregate([
    {"$unwind": "$transactions"},
    {"$sort": {"transactions.selling_price": -1}},
    {"$limit": 1}
])

for apartment in result:
    print("Korkein myyntihinta:", apartment["transactions"]["selling_price"])

client.close()
```

Kysymysten vastaukset:

- Monessako asunnossa postinumero on 00700? 17
- Monessako asunnossa rakennusvuosi on 2000-luvulla? 324
- Monessako asunnossa pinta-ala on välillä 50–70 m^2? 150
- Montako asuntoa myytiin vuosina 2010–2012? 673
- Mikä on kallein asunnon myyntihinta aineistossa? 769290

Tehtävä 7

Tehtävän pistemäärä: /10 pistettä

Toteutettu koodi:

```
import csv
import sqlite3
from pathlib import Path

csv_files = [Path("csv") / f"2023-{m:02d}.csv" for m in range(4, 11)]

for file in csv_files:
    if not file.exists():
        raise FileNotFoundError(file)

connection = sqlite3.connect("bikes_2023.db")

connection.executescript("""
    DROP TABLE IF EXISTS Trips;
    DROP TABLE IF EXISTS Stations;

    CREATE TABLE Stations (id INTEGER PRIMARY KEY, name TEXT);
    CREATE TABLE Trips (
        id INTEGER PRIMARY KEY,
        start_time TEXT,
        end_time TEXT,
        start_station_id INTEGER REFERENCES Stations,
        end_station_id INTEGER REFERENCES Stations,
        distance INTEGER,
        duration INTEGER
    );"""
)

trip_sql = """INSERT INTO Trips
              (start_time, end_time, start_station_id, end_station_id, distance, duration)
              VALUES (?, ?, ?, ?, ?, ?)"""

stations = {}
trips = []
```

```
try:
    for file in csv_files:
        with open(file, encoding="utf-8-sig", newline="") as f:
            for row in csv.DictReader(f):
                if (
                    not row["Covered distance (m)"].strip()
                    or not row["Duration (sec.)"].strip()
                ):
                    continue

                start_id = int(row["Departure station id"])
                end_id = int(row["Return station id"])

                stations.setdefault(start_id, row["Departure station name"])
                stations.setdefault(end_id, row["Return station name"])

                trips.append((
                    row["Departure"], row["Return"], start_id, end_id,
                    int(float(row["Covered distance (m)"])),
                    int(float(row["Duration (sec.)"]))
                ))

                if len(trips) >= 10000:
                    connection.executemany(trip_sql, trips)
                    trips.clear()

    connection.executemany(trip_sql, trips)
    connection.executemany(
        "INSERT INTO Stations VALUES (?, ?)", stations.items()
    )
    connection.commit()

    for table in ("Stations", "Trips"):
        count = connection.execute(
            f"SELECT COUNT(*) FROM {table}"
        ).fetchone()[0]
        print(f"{table}: {count}")

    print("Station 100:", connection.execute(
        "SELECT * FROM Stations WHERE id = 100"
    ).fetchone())

    print("Trip 123456:", connection.execute(
        "SELECT * FROM Trips WHERE id = 123456"
    ).fetchone())

finally:
    connection.close()
```

Tehtävä 8

Tehtävän pistemäärä: /10 pistettä

Lähteet:

1. https://tikape.mooc.fi/syksy-2026/materiaali/
2. https://www.sqlite.org/docs.html
3. https://docs.python.org/3/
4. https://www.mongodb.com/docs/manual/

Tekoälyn käyttö

Projektin aikana tekoälyä hyödynnettiin tiedonhaun ja ongelmanratkaisun tukena. Google-haun kautta käytetty Gemini tarjosi tarvittaessa lisätietoa ja ehdotuksia työn edistämiseksi.

Liitteet

Koodi löytyy git-reposta: https://github.com/dopamiini/tikape-project